

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Basic SQL querying (select and project) using where clause

- In SQL, the WHERE clause is used to filter rows based on specific conditions.
- Whether you are retrieving, updating, or deleting data, WHERE ensures that only relevant records are affected.
- The WHERE clause helps you:
 - Filter rows that meet certain conditions
 - Target specific data using logical, comparison and pattern-based operators

Basic SQL querying (select and project) using where clause

```
□ CREATE TABLE Emp1 (  
  EmpID INT PRIMARY KEY,  
  Name VARCHAR(50),  
  Country VARCHAR(50),  
  Age INT,  
  Mob VARCHAR(15)  
);
```

```
INSERT INTO Emp1 VALUES  
(1, 'Shubham', 'India', 23, '738479734'),  
(2, 'Aman', 'Australia', 21, '436789555'),  
(3, 'Naveen', 'Sri Lanka', 24, '34873847'),  
(4, 'Aditya', 'Austria', 21, '328440934'),  
(5, 'Nishant', 'Spain', 22, '73248679');
```

```
SELECT * FROM Emp1;
```

Basic SQL querying (select and project) using where clause

EmpID	Name	Country	Age	mob
1	Shubham	India	23	738479734
2	Aman	Australia	21	436789555
3	Naveen	Sri lanka	24	34873847
4	Aditya	Austria	21	328440934
5	Nishant	Spain	22	73248679

Basic SQL querying (select and project) using where clause

- **Example 1: Where Clause with Logical Operators**

- `SELECT * FROM Emp1 WHERE Age=24;`

- **Example 2: WHERE with Comparison Operators**

- `SELECT EmpID, Name, Country FROM Emp1 WHERE Age > 21;`

- **Example 3: WHERE Clause with UPDATE Statement**

- `UPDATE Emp1 set Age=26 where NAME = 'Aman';`

Basic SQL querying (select and project) using where clause

- **Example 4: Where Clause with BETWEEN Operator**
- The BETWEEN operator is used to filter records within a **specified range**, and it includes both the start and end values.
 - `SELECT * FROM Emp1
WHERE Age BETWEEN 22 AND 24;`

Basic SQL querying (select and project) using where clause

- **Example 5: Where Clause with LIKE Operator**
- It is used to fetch filtered data by searching for a particular pattern in the where clause.
- The '%' (wildcard) signifies the later characters here which can be of any length and value.
 - `SELECT * FROM Emp1 WHERE Name LIKE 'S%';`
 - `SELECT * FROM Emp1 WHERE Name LIKE '%m%';`

Basic SQL querying (select and project) using where clause

- **Example 6: Where Clause with IN Operator/ NOT IN Operator**
- It is used to fetch the filtered data same as fetched by '=' operator just the difference is that here we can specify multiple values for which we can get the result set.
 - `SELECT Name FROM Emp1 WHERE Age IN (21,23);`
 - `SELECT Name FROM Emp1 WHERE Age NOT IN (21,23);`

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, **arithmetic** & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Arithmetic Operators



- SQL Arithmetic Operators
- SQL Comparison Operators

Arithmetic Operators



- SQL Arithmetic Operators
- SQL Comparison Operators

Arithmetic Operators

- The SQL Arithmetic operators are used to perform basic mathematical operations on numeric data types in a database.
- The most common arithmetic operators used in SQL are:

+	Adds the values both sides of the operator.
-	Subtracts right hand operand from left hand operand.
*	Multiplies the values from both sides of the operator.
/	Divides left hand operand by right hand operand.
%	Divides left hand operand by right hand operand and returns the remainder remainder.

Arithmetic Operators

- ❑ CREATE TABLE EMPLOYEE
- ❑ (ID INT NOT NULL,
NAME VARCHAR(15) NOT NULL,
AGE INT NOT NULL,
ADDRESS CHAR(25),
SALARY DECIMAL(10, 2),
PRIMARY KEY(ID));

Arithmetic Operators

ID	NAME	AGE	ADDRESS	SALARY
1	khilan	22	Nijamabad	57500.84
4	Kaushik	18	Bangalore	47275.43
5	chaitali	23	Ranchi	40700.76
6	Hardhik	19	Noida	44200.09
3	komal	23	Chennai	44200.09
2	Ramesh	21	Hyderabad	25550.12

Arithmetic Operators

- `SELECT NAME, SALARY, SALARY + 5000 AS "SALARY_BONUS"`
`FROM Employee;`

	NAME	SALARY	SALARY_BONUS
2	Kaushik	47275.43	52275.43
3	chaitali	40700.76	45700.76
4	Hardhik	44200.09	49200.09
5	komal	44200.09	49200.09
6	Ramesh	25550.12	30550.12

Arithmetic Operators

- `SELECT NAME, SALARY, SALARY - 5000 AS "SALARY_Decrement" FROM Employee;`

	NAME	SALARY	SALARY_DECREMEI
1	khilan	57500.84	52500.84
2	Kaushik	47275.43	42275.43
3	chaitali	40700.76	35700.76
4	Hardhik	44200.09	39200.09
5	komal	44200.09	39200.09

Arithmetic Operators

- `SELECT NAME, AGE, ADDRESS, SALARY * 1.10 AS new_salary_with_bonus FROM Employee;`

	NAME	AGE	ADDRESS	NEW_SALARY_WITH
1	khilan	22	Nijamabad	63250.924
2	Kaushik	18	Bangalore	52002.973
3	chaitali	23	Ranchi	44770.836
4	Hardhik	19	Noida	48620.099
5	komal	23	Chennai	48620.099

Arithmetic Operators

- `SELECT NAME, AGE, ADDRESS, SALARY / 10 AS new_salary_with_Division FROM Employee;`
- `SELECT * FROM Employee where MOD(ID,2)=0;`

Arithmetic

- SQL Arithmetic Operators
- SQL Comparison Operators

Comparison Operators

- SQL Comparison Operators are used to compare two values and filter the result set based on a specific condition. Used in the WHERE clause to filter records based on conditions.

=	The SQL Equal Operator checks if the values of two operands are equal.
!=	The SQL Not Equal Operator checks if the values of two operands are not equal.
>=	The SQL Greater Than Equals to Operator checks if the value of the left operand is greater than or equal to the value of the right operand.
<	The SQL Less Than Operator checks if the value of the left operand is less than the value of the right operand.
>	The SQL Greater Than Operator checks if the value of the left operand is greater than the value of the right operand.
<=	The SQL Less Than Equals to Operator checks if the value of the left operand is less than or equal to the value of the right operand.

Comparison Operators

- ❑ `CREATE TABLE MATHS( ROLL_NUMBER INT, S_NAME VARCHAR(10), MARKS INT);`
- ❑ `INSERT INTO MATHS (Roll_Number, S_name, marks)
VALUES (1, 'ABHI', 70), (2, 'RAVI', 80), (3, 'ARJUN',
90), (4, 'SAM', 100), (5, 'MOHAN', 50), (6,
'ROHAN', 10), (7, 'ROCKY', 20), (8, 'AYUSH', 40),
(9, 'NEHA', 30), (10, 'KRITI', 60);`
- ❑ `SELECT * FROM MATHS;`

Comparison Operators

- ❑ `SELECT * FROM MATHS WHERE MARKS=50;`
- ❑ `SELECT * FROM MATHS WHERE MARKS>60;`
- ❑ `SELECT * FROM MATHS WHERE MARKS<40;`
- ❑ `SELECT * FROM MATHS WHERE MARKS>=80;`
- ❑ `SELECT * FROM MATHS WHERE MARKS<=30;`
- ❑ `SELECT * FROM MATHS WHERE MARKS<>70;`
- ❑ `SELECT * FROM MATHS WHERE MARKS!=100;`

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & **logical operations**, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Logical operations

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the subquery values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
SOME	TRUE if any of the subquery values meet the condition

Comparison Operators

- ❑ `CREATE TABLE EMP3(EMP_ID INT, EMP_NAME VARCHAR(50), EMP_CITY VARCHAR(20), EMP_Country VARCHAR(20));`
- ❑ `INSERT INTO EMP3 (EMP_ID, EMP_NAME, EMP_CITY, EMP_Country) VALUES (101, 'Utkarsh Tripathi', 'Varanasi', 'India'), (102, 'Abhinav Singh', 'Varanasi', 'India'), (103, 'Utkarsh Raghuvamsi', 'Varanasi', 'India'), (104, 'Utkarsh Singh', 'Allahabad', 'India'), (105, 'Sudhanshu Yadav', 'Allahabad', 'India'), (106, 'Ashutosh Kumar', 'Pune', 'India');`

❑

Logical operations

EMP_ID	EMP_NAME	EMP_CITY	EMP_COUNTRY
101	Utkarsh Tripathi	Varanasi	India
102	Abhinav Singh	Varanasi	India
103	Utkarsh Raghuvamsi	Varanasi	India
104	Utkarsh Singh	Allahabad	India
105	Sudhanshu Yadav	Allahabad	India
106	Ashutosh Kumar	Pune	India

Logical operations

❑ 1. AND Operator

- ❑ The AND operator is used to combine two or more conditions in an SQL query.
- ❑ It returns records only when all conditions specified in the query are true.
- ❑ This operator is commonly used when filtering data that must satisfy multiple criteria simultaneously.

Logical operations

- `SELECT * FROM Emp3 WHERE emp_city = 'Allahabad' AND emp_country = 'India';`

EMP_ID	EMP_NAME	EMP_CITY	EMP_COUNTRY
104	Utkarsh Singh	Allahabad	India
105	Sudhanshu Yadav	Allahabad	India

Logical operations

□ 2. IN Operator

- The IN operator streamlines checking **whether a value matches any value from a specified list**, making queries more concise and readable than using multiple OR conditions.
- It is especially useful for filtering results by several possible values in a column, helping reduce query complexity.

Logical operations

- `SELECT * FROM Emp3 WHERE emp_city IN ('Allahabad', 'Patna');`

EMP_ID	EMP_NAME	EMP_CITY	EMP_COUNTRY
104	Utkarsh Singh	Allahabad	India
105	Sudhanshu Yadav	Allahabad	India
106	Ashutosh Kumar	Pune	India

Logical operations

□ **3. NOT Operator**

- The NOT operator is used to reverse the result of a condition, returning TRUE when the condition is FALSE.
- It is typically used to exclude records that match a specific condition, making it useful for filtering out unwanted data.

Logical operations

- **Example:** Retrieve the records of employees from the employee table whose city names do not start with the letter 'A'.
- **Query:** `SELECT * FROM Emp3 WHERE emp_city NOT LIKE 'A%';`

EMP_ID	EMP_NAME	EMP_CITY	EMP_COUNTRY
101	Utkarsh Tripathi	Varanasi	India
102	Abhinav Singh	Varanasi	India
103	Utkarsh Raghuvamsi	Varanasi	India
106	Ashutosh Kumar	Pune	India

Logical operations

□ **OR Operator**

- The OR operator combines multiple conditions in a SQL query and returns TRUE if at least one of the conditions is satisfied.
- It is ideal for situations where you want to retrieve records that meet any of several possible conditions.

□ **Example**

- Retrieve the records of employees from the employee table who are either from 'Varanasi' or have 'India' as their country.

Logical operations

- `SELECT * FROM Emp3 WHERE emp_city = 'Varanasi' OR emp_country = 'India';`

EMP_ID	EMP_NAME	EMP_CITY	EMP_COUNTRY
101	Utkarsh Tripathi	Varanasi	India
102	Abhinav Singh	Varanasi	India
103	Utkarsh Raghuvamsi	Varanasi	India
104	Utkarsh Singh	Allahabad	India
105	Sudhanshu Yadav	Allahabad	India
106	Ashutosh Kumar	Pune	India

Logical operations

□ 5. LIKE Operator

- The LIKE operator in SQL is used in the WHERE clause to search for a specified pattern in a column.
- The **LIKE** operator works with two main wildcards:
 - %: Represents zero or more characters. It allows matching any sequence of characters in the string.
 - _: Represents exactly one character. It is used when you want to match a specific number of characters at a given position.
- **Example**
- Retrieve the records of employees from the employee table whose city names start with the letter 'P'.

Logical operations

- `SELECT * FROM Emp3 WHERE emp_city LIKE 'P%';`

EMP_ID	EMP_NAME	EMP_CITY	EMP_COUNTRY
106	Ashutosh Kumar	Pune	India

Logical operations

❑ **BETWEEN Operator**

- ❑ The BETWEEN operator in SQL allows us to test if a value or expression lies within a **specified range**.
 - ❑ **Inclusive Range** – Includes both the lower and upper limits in the result set.
 - ❑ **Versatile Use** – Works with numbers, dates, and text values.
- ❑ **Example**
 - ❑ Retrieve the records of employees from the employee table whose emp_id values fall within the range of 101 to 104 (inclusive).

Logical operations

- `SELECT * FROM Emp3 WHERE emp_id BETWEEN 101 AND 104;`

EMP_ID	EMP_NAME	EMP_CITY	EMP_COUNTRY
101	Utkarsh Tripathi	Varanasi	India
102	Abhinav Singh	Varanasi	India
103	Utkarsh Raghuvamsi	Varanasi	India
104	Utkarsh Singh	Allahabad	India

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

SQL functions(Date and Time, Numeric, String conversion)

- **Scalar functions** are the **built-in functions in SQL**, and whatever be the input provided to the scalar functions, the **output returned by these functions will always be a single value.**
- In SQL, **each record is operated independently** by the scalar functions.

SQL functions(Date and Time, Numeric, String conversion)

- **Date & Time Functions**
- String Functions
- Number Functions

SQL functions(Date and Time, Numeric, String conversion)

- **SYSDATE:** Returns the current date and time from the database server.

- SELECT SYSDATE FROM DUAL;

	SYSDATE
1	8/19/2025, 6:16:10

- **CURRENT_DATE / CURRENT_TIMESTAMP:** Returns the current date or timestamp, often including timezone information.

- SELECT CURRENT_DATE FROM DUAL;

- SELECT CURRENT_TIMESTAMP FROM DUAL;

CURRENT_DATE
8/19/2025, 6:17:14

CURRENT_TIMESTAMP
2025-08-19T06:17:49.226484Z

SQL functions(Date and Time, Numeric, String conversion)

- ❑ **EXTRACT(part FROM date):** Extracts a specific part (e.g., YEAR, MONTH, DAY, HOUR, MINUTE, SECOND) from a date or timestamp.
 - ❑ SELECT EXTRACT(YEAR FROM SYSDATE) AS current_year FROM DUAL;

CURRENT_YEAR
2025

- ❑ **ADD_MONTHS(date, n):** Adds or subtracts a specified number of months to a date.
 - ❑ SELECT ADD_MONTHS(SYSDATE, 3) AS date_plus_3_months FROM DUAL;

DATE_PLUS_3_MONTHS
11/19/2025, 6:21:46 AM

SQL functions(Date and Time, Numeric, String conversion)

- **LAST_DAY(date):** Returns the last day of the month for a given date.

- `SELECT LAST_DAY(SYSDATE) AS last_day_of_month FROM DUAL;`

LAST_DAY_OF_MONTH
8/31/2025, 6:24:35 AM

- **NEXT_DAY(date, 'day_of_week'):** Returns the date of the next specified day of the week following a given date.

- `SELECT NEXT_DAY(SYSDATE, 'MONDAY') AS next_monday FROM DUAL;`

NEXT_MONDAY
8/25/2025, 6:25:17

SQL functions(Date and Time, Numeric, String conversion)

- **MONTHS_BETWEEN(date1, date2):** Returns the number of months between two dates.

- `SELECT MONTHS_BETWEEN('01-JAN-2025', '01-JUL-2024') AS months_diff FROM DUAL;`

MONTHS_DIFF
6

- **TRUNC(date, 'format_model'):** Truncates a date to a specified unit (e.g., 'YEAR', 'MONTH', 'DD').

- `SELECT TRUNC(SYSDATE, 'MONTH') AS first_day_of_month FROM DUAL;`

FIRST_DAY_OF_MONTH
8/1/2025, 12:00:00 AM

SQL functions(Date and Time, Numeric, String conversion)

- **ROUND(date, 'format_model')**: Rounds a date to the nearest specified unit.

- `SELECT ROUND(SYSDATE, 'YEAR') AS rounded_to_year FROM DUAL;`

ROUNDED_TO_YEAR
1/1/2026, 12:00:00 AM

- **TO_CHAR(date, 'format_model')**: Converts a date to a character string with a specified format.

- `SELECT TO_CHAR(SYSDATE, 'DD-MON-YYYY HH24:MI:SS') AS formatted_date FROM DUAL;`

FORMATTED_DATE
19-AUG-2025 06:32:18

SQL functions(Date and Time, Numeric, String conversion)

- Date Functions
- String Functions
- Number Functions

SQL functions(Date and Time, Numeric, String conversion)-Number Functions

- Some of the commonly used Number functions in SQL includes:

Number Functions	
CEIL()	MOD()
FLOOR()	POWER(m, n)
ABS()	SIGN()
GREATEST()	SQRT()
LEAST()	RAND()

SQL functions(Date and Time, Numeric, String conversion)-Number Functions

- **CEIL():** It returns the smallest integer value that is greater than or equal to a number.
 - **Syntax:** SELECT CEIL(25.75);
 - **Output:** 26

- **FLOOR():** It returns the largest integer value that is less than or equal to a number.
 - **Syntax:** SELECT FLOOR(25.75);
 - **Output:** 25

SQL functions(Date and Time, Numeric, String conversion)-Number Functions

- **ABS() – Absolute Value**
- The **ABS() function** returns the absolute value of a number, which is the number without its sign (i.e., it converts negative numbers to positive).
 - **Syntax:** SELECT ABS(-25);
 - **Output:** 25
- **GREATEST():** It returns the greatest value in a list of expressions.
 - **Syntax:** SELECT GREATEST(30, 2, 36, 81, 125);
 - **Output:** 125

SQL functions(Date and Time, **Numeric**, String conversion)-Number Functions

- **LEAST():** It returns the smallest value in a list of expressions.
 - **Syntax:** SELECT LEAST(30, 2, 36, 81, 125);
 - **Output:** 2
- **MOD():** It returns the remainder (aka. modulus) of n divided by m.
 - **Syntax:** SELECT MOD(18, 4);
 - **Output:** 2

SQL functions(Date and Time, Numeric, String conversion)-Number Functions

- ❑ **POWER(m, n):** It returns m raised to the nth power.
 - ❑ **Syntax:** SELECT POWER(4, 2);
 - ❑ **Output:** 16

- ❑ **SIGN():** It returns a value indicating the sign of a number. A return value of 1 means positive; 0 means negative.
 - ❑ **Syntax:** SELECT SIGN(255.5);
 - ❑ **Output:** 1

SQL functions(Date and Time, Numeric, String conversion)-Number Functions

- **SQRT():** It returns the square root of a number.
 - **Syntax:** SELECT SQRT(25);
 - **Output:** 5
- **RAND() – Random Number**
- The **RAND()** function generates a random floating-point number between 0 and 1. This function is commonly used for simulations, lotteries, or generating random samples.
 - **Syntax:** SELECT DBMS_RANDOM.Value() FROM DUAL;
 - **Syntax:** SELECT DBMS_RANDOM.Value(1,100) FROM DUAL;
 - **Output:** 0.287372

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- Date Functions
- String Functions
- Number Functions

SQL functions(Date and Time, Numeric, String conversion)-String Functions

L_NUMBER	F_NAME	L_NAME	STREET	CITY	STATE
1	Arun	Ravi	DNR	BVRM	AP
2	Ajay	Kiran	JP Road	BVRM	AP
3	Satish	Sandeep	MG Road	BZA	AP
4	Sandhya	Rani	KrishnaLanka	BZA	AP
5	Geetha	Manaswi	BesantRoad	BZA	AP

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- ❑ **LENGTH():** This string function returns the length of the specified word. It shows the number of characters from the word.
- ❑ **Syntax:**
 - ❑ `SELECT LENGTH(Column_Name) as Alias_Name
FROM Table_Name;`
- ❑ **Example:**
 - ❑ `SELECT F_Name,LENGTH(F_Name)as Lenght_Of_Name
FROM Student1;`

SQL functions(Date and Time, Numeric, String conversion)-String Functions

	F_NAME	LENGHT_OF_NAME
1	Arun	4
2	Ajay	4
3	Satish	6
4	Sandhya	7
5	Geetha	6

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **CONCAT():** This string function concatenates two strings or words and forms a new string in the result.
- **Syntax:**
 - **SELECT** CONCAT(Column_Name1, Column_Name2, column_NameN) **AS** Alias_Name **FROM** Table_Name;
- **Example:**
 - **SELECT** CONCAT(F_Name, ' ', L_Name) **AS** fullName **FROM** Student1;

FULLNAME
Arun Ravi
Ajay Kiran
Satish Sandeep
Sandhya Rani
Geetha Manaswi

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- ❑ **SUBSTRING():**
- ❑ The SUBSTRING SQL string function extracts a substring or a portion of a string from a larger string.
- ❑ **It takes three arguments:** the string to be removed from, the substring's starting position, and the substring's length.
- ❑ **Syntax** SELECT substr(L_Name,1,3) AS SUBStringExample FROM Student1;

SUBSTRING
Rav
Kir
San
Ran
Man

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **UPPER():** It converts the value of a field to uppercase.

- **Syntax:**

- `SELECT UPPER(column_name) FROM table_name;`

- **Example:**

- `SELECT L_Name,Upper(L_Name) AS UCASE_Name FROM Student1;`

L_NAME	UCASE_NAME
Ravi	RAVI
Kiran	KIRAN
Sandeep	SANDEEP
Rani	RANI
Manaswi	MANASWI

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **LOWER():** It converts the value of a field to lowercase.

- **Syntax:**

- `SELECT LOWER(column_name) FROM table_name;`

- **Example:**

- `SELECT L_Name, Lower(L_Name) AS LCASE_Name FROM Student1;`

L_NAME	LCASE_NAME
Ravi	ravi
Kiran	kiran
Sandeep	sandeep
Rani	rani
Manaswi	manaswi

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- ❑ **INITCAP(string):** Converts the first letter of each word to uppercase and the rest to lowercase.
- ❑ **Syntax:**
 - ❑ `SELECT Initcap(column_name) FROM table_name;`
- ❑ **Example:**
- ❑ `SELECT L_Name,Initcap(L_Name) AS INITCap_Name FROM Student1;`

L_NAME	INITCAP_NAME
Ravi	Ravi
Kiran	Kiran
Sandeep	Sandeep
Rani	Rani
Manaswi	Manaswi

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **REVERSE():** The REVERSE() function reverses the characters in a string. It's useful in situations where we need to process data backward, such as for password validation or certain pattern matching.
 - **Syntax:** **SELECT** REVERSE(Column_Name) **AS** Alias_Name **FROM** Table_Name;
 - **Example:** **SELECT** F_Name,reverse(F_Name) **AS** Reverse_Name **FROM** Student1;

F_NAME	REVERSE_NAME
Arun	nurA
Ajay	yajA
Satish	hsitaS
Sandhya	ayhdnaS
Geetha	ahteeG

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **ASCII():** Converts a single character string to its corresponding ASCII code, between 0 and 255. If the character expression evaluates to multiple characters, the ASCII code corresponding to the first character in the expression is returned.
 - **Syntax:** Ascii(column_name);
- **Example:** select F_Name, ascii(F_Name) as Aciiformat from Student1;

F_NAME	ACIIFORMAT
Arun	65
Ajay	65
Satish	83
Sandhya	83
Geetha	71

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **REPLACE():** In Oracle SQL, the REPLACE function is used to substitute all occurrences of a specified substring within a string with another substring.
 - **Syntax:** REPLACE(original_string, search_string, replacement_string)**Example:**
- **Example:** select F_Name, REPLACE(F_Name,'A','K') as StringRepalce from Student1;

F_NAME	STRINGREPALCE
Arun	Krun
Ajay	Kjay
Satish	Satish
Sandhya	Sandhya
Geetha	Geetha

SQL functions(Date and Time, Numeric, String conversion)-String Functions

- **INSTR():** The INSTR() function returns the position of the first occurrence of a substring within a string. If the substring is not found, it returns 0. It is useful for finding where specific text appears in a larger string.
- **Syntax:** Instr(Column_Name, 'character');
- **Example:** select F_Name, instr(F_Name,'a') as INSTRFORMAT from Student1;

F_NAME	INSTRFORMAT
Arun	0
Ajay	3
Satish	2
Sandhya	2
Geetha	6

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).**Creating tables with relationship, implementation of key and integrity constraints,** nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Creating tables with relationship, implementation of key and integrity constraints

- *** Write the syntax for creating primary key and foreign key ***

UNIT III:

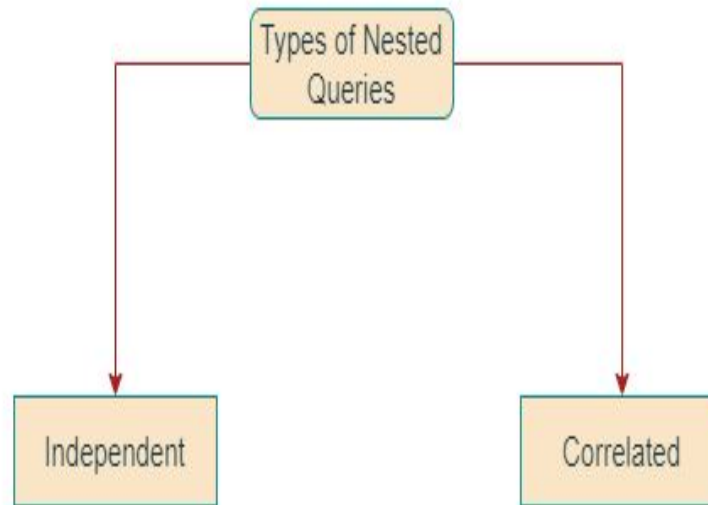
- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, **nested queries, sub queries**, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Nested queries, Sub queries

- In the context of SQL, the terms "nested query" and "subquery" are often used interchangeably, as they refer to the same concept: a query embedded within another SQL query.
 - `SELECT column_name(s) FROM table_name WHERE column_name IN (SELECT column_name FROM another_table WHERE condition);`
- In this example, `(SELECT column_name FROM another_table WHERE condition)` is the subquery.
- It is "nested" within the WHERE clause of the outer SELECT statement.

Nested queries, Sub queries

- Types of Nested Queries:



Nested queries, Sub queries

- **Independent Nested Queries:**
- In independent nested queries, **the execution order is from the innermost query to the outer query.**
- An **outer query won't be executed until its inner query completes** its execution.
- The **outer query uses the result of the inner query.**
- Operators such as **IN, NOT IN, ALL, and ANY** are used to write independent nested queries.

Nested queries, Sub queries

- ❑ **Co-related Nested Queries:**
- ❑ In co-related nested queries, **the inner query uses the values from the outer query to execute the inner query for every row processed by the outer query.**
- ❑ The co-related nested queries **run slowly** because the inner query is executed for every row of the outer query's result.
- ❑ Operators such as **Exists, Not Exists, ALL,** and **ANY** are used to write independent nested queries.

Nested queries, Sub queries

- Create table Student4(ID int PRIMARY KEY,
- SName varchar(20),
- City Varchar(20),
- Age int);

Nested queries, Sub queries

- ❑ INSERT INTO Student4(ID, SName, City, Age)
VALUES
- ❑ (1, 'Aruna', 'Delhi', 19),
- ❑ (2, 'Bhanu', 'Delhi', 19),
- ❑ (3, 'Chandu', 'BVRM', 20),
- ❑ (4, 'Dhanush', 'BZA', 21),
- ❑ (5, 'Sandhya', 'Mumbai', 19);

Nested queries, Sub queries

ID	SNAME	CITY	AGE
1	Aruna	Delhi	19
2	Bhanu	Delhi	19
3	Chandu	BVRM	20
4	Dhanush	BZA	21
5	Sandhya	Mumbai	19

Nested queries, Sub queries

- ❑ create table Course1 (CID varchar(10) Primary Key, CName Varchar(30));
- ❑ insert into Course1 (CID, Cname)values
- ❑ ('C1', 'DBMS'),
- ❑ ('C2', 'OOPS'),
- ❑ ('C3','CO');

CID	CNAME
C1	DBMS
C2	OOPS
C3	CO

Nested queries, Sub queries

- ❑ create table Student_Course(ID int, CID varchar(10), Foreign Key(ID) references Student4(ID), Foreign Key(CID)references Course1 (CID));
- ❑ insert into Student_Course(ID,CID) VALUES
- ❑ (1,'C1'),(1,'C3'),(2,'C1'),(3,'C2'),(4,'C2'),(4,'C3');

Nested queries, Sub queries

- `Select * from Student_Course;`

ID	CID
1	C1
1	C3
2	C1
3	C2
4	C2
4	C3

Nested queries, Sub queries: **IN**

- Find the ID's of students who are enrolled in the courses 'DBMS' or 'CO'?
- `SELECT CID FROM COURSE1 WHERE CNAME IN ('DBMS','CO');`

CID
C1
C3

Nested queries, Sub queries: **IN**

- `SELECT ID,CID FROM STUDENT_COURSE WHERE CID IN`
- `(SELECT CID FROM COURSE1 WHERE CNAME IN ('DBMS','CO'));`

ID	CID
1	C1
1	C3
2	C1
4	C3

Nested queries, Sub queries: NOT IN

- SELECT ID,CID FROM STUDENT_COURSE WHERE CID NOT IN
- (SELECT CID FROM COURSE1 WHERE CNAME IN ('DBMS','CO'));

ID	CID
3	C2
4	C2

Nested queries, Sub queries: **Exists**

- Find the names of students who are enrolled in the course 'C1':
- `SELECT SNAME FROM STUDENT4 S WHERE EXISTS (
SELECT * FROM STUDENT_COURSE SC
WHERE S.ID = SC.ID AND SC.CID = 'C1');`

SNAME
Aruna
Bhanu

Nested queries, Sub queries: **Not Exists**

- `SELECT SNAME FROM STUDENT4 S WHERE NOT EXISTS (
SELECT * FROM STUDENT_COURSE SC
WHERE S.ID = SC.ID AND SC.CID = 'C1');`

SNAME
Dhanush
Sandhya
Chandu

Nested queries, Sub queries: ANY

- SQL ANY compares a value of the first table with all values of the second table and **returns the row if there is a match with any** value.
- Syntax: `SELECT column FROM table1 WHERE column OPERATOR ANY ( SELECT column FROM table2 );`

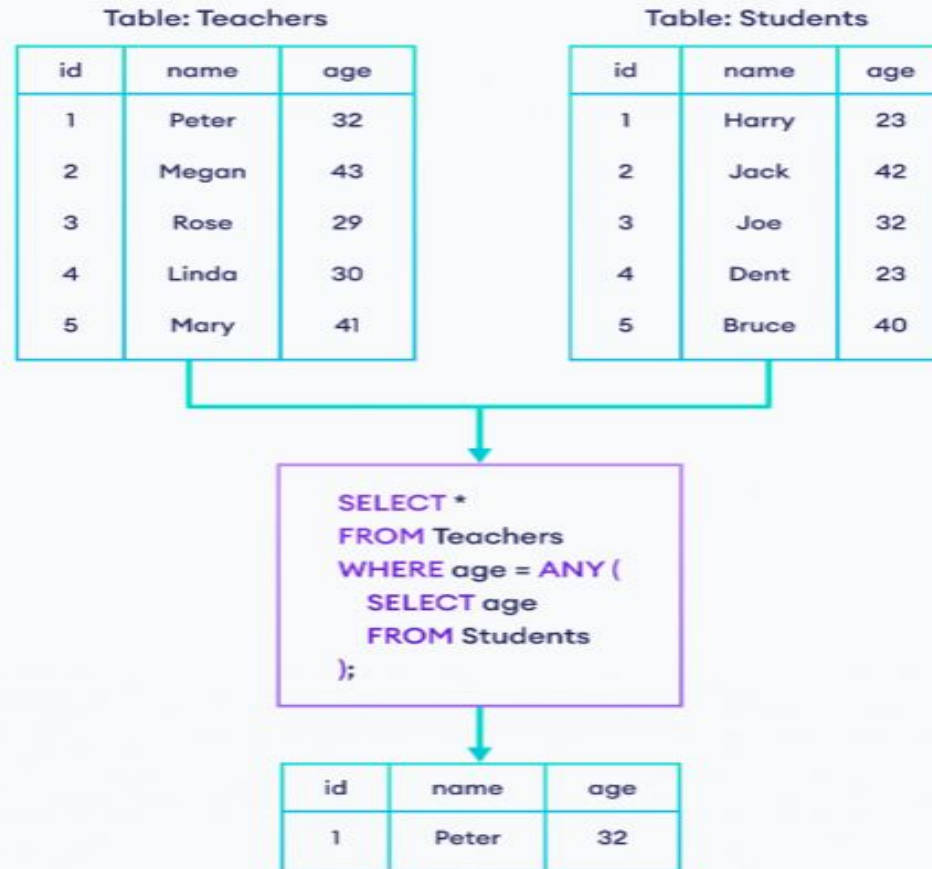
Nested queries, Sub queries: **ANY**

- Find teachers whose age is similar to any of the student's age:

Table: Teachers			Table: Students		
id	name	age	id	name	age
1	Peter	32	1	Harry	23
2	Megan	43	2	Jack	42
3	Rose	29	3	Joe	32
4	Linda	30	4	Dent	23
5	Mary	41	5	Bruce	40

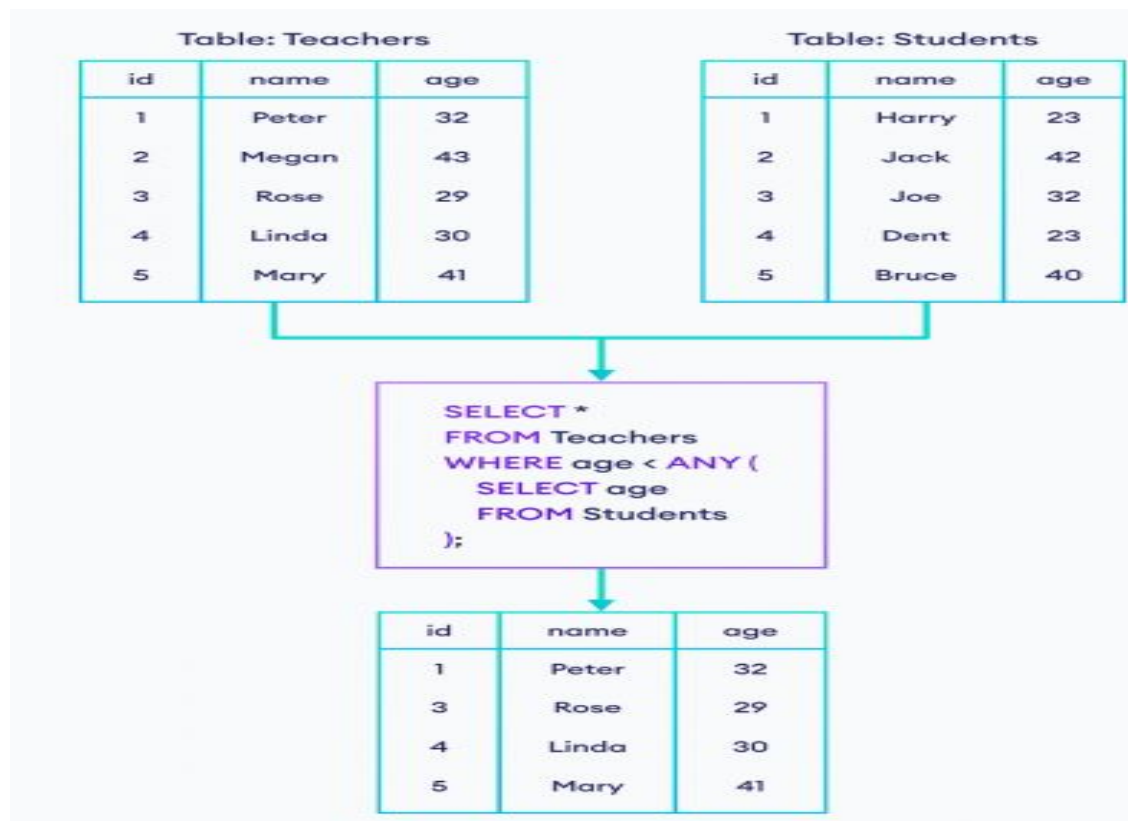
- `SELECT * FROM Teachers WHERE age = ANY ( SELECT age FROM Students );`

Nested queries, Sub queries: ANY



Nested queries, Sub queries: **ANY**

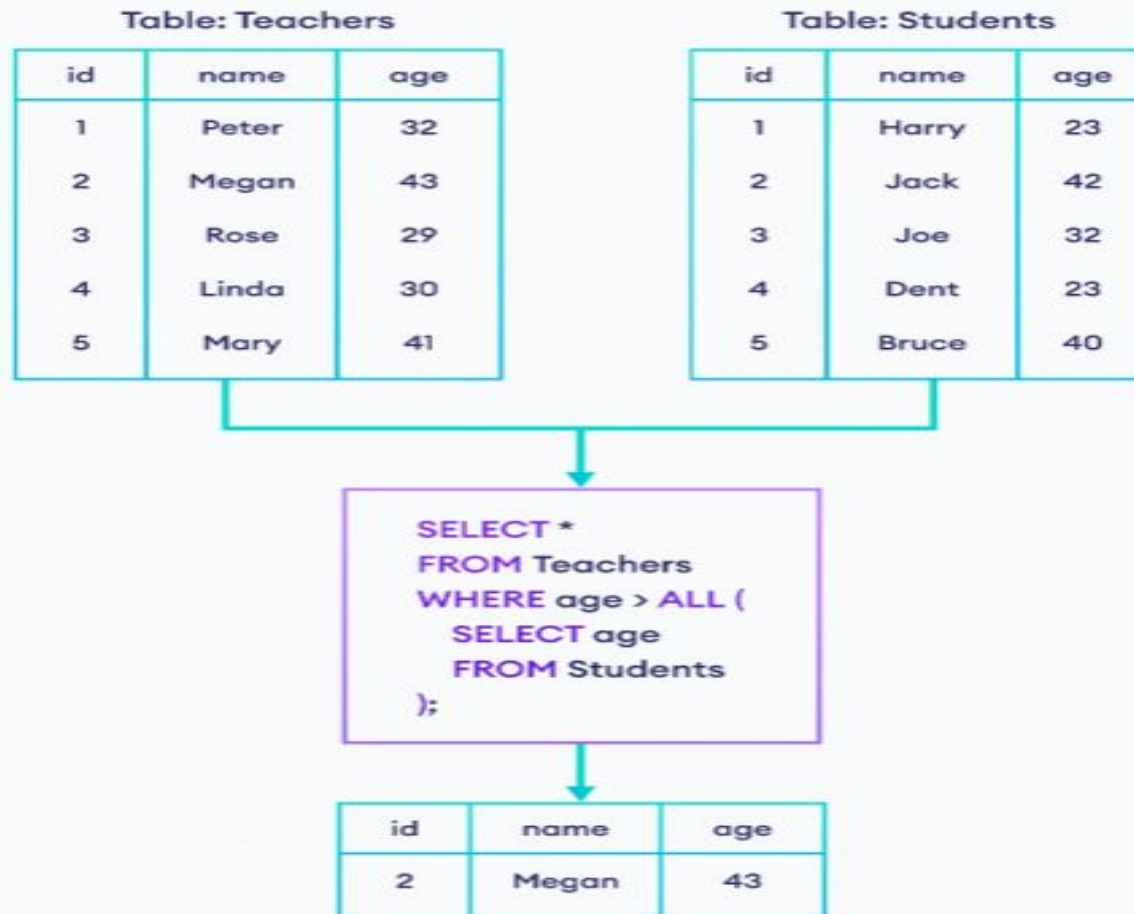
- Find teachers whose age is less than any student.



Nested queries, Sub queries: **ALL**

- SQL ALL compares a value of the first table with all values of the second table and **returns the row if there is a match with all** values.
- Syntax: SELECT column FROM table1 WHERE column OPERATOR **ALL** (SELECT column FROM table2);
- Find teachers whose age is greater than all students?
- SELECT * FROM Teachers WHERE age > ALL (SELECT age FROM Students);

Nested queries, Sub queries: **ALL**



UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, **grouping**, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Grouping

- The GROUP BY statement **groups rows that have the same values in specified columns**. It is widely used for creating summaries, like sales by region or users by age group.
 - Used with SELECT statement.
 - Comes after WHERE, before HAVING and ORDER BY.
 - **Query order:** SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY.
- **Syntax:** SELECT column1, function_name(column2)
FROM table_name GROUP BY column1, column2

Grouping

- ❑ CREATE TABLE Emp2 (
emp_no INT PRIMARY KEY,
name VARCHAR(50),
sal DECIMAL(10,2),
age INT
);

Grouping

- ❑ INSERT INTO Emp2 (emp_no, name, sal, age) VALUES
(1, 'Aarav', 50000.00, 25),
(2, 'Aditi', 60000.50, 30),
(3, 'Aarav', 75000.75, 35),
(4, 'Anjali', 45000.25, 28),
(5, 'Chetan', 80000.00, 32),
(6, 'Divya', 65000.00, 27),
(7, 'Gaurav', 55000.50, 29),
(8, 'Divya', 72000.75, 31),
(9, 'Gaurav', 48000.25, 26),
(10, 'Divya', 83000.00, 33);

Grouping

EMP_NO	NAME	SAL	AGE
1	Aarav	50000	25
2	Aditi	60000.5	30
3	Aarav	75000.75	35
4	Anjali	45000.25	28
5	Chetan	80000	32
6	Divya	65000	27
7	Gaurav	55000.5	29
8	Divya	72000.75	31
9	Gaurav	48000.25	26

Grouping

- `SELECT name, SUM(sal) FROM Emp2 GROUP BY name;`

NAME	SUM(SAL)
Aarav	125000.75
Aditi	60000.5
Anjali	45000.25
Chetan	80000
Divya	220000.75
Gaurav	103000.75

Grouping

- `SELECT NAME, SUM(sal) FROM Emp2 GROUP BY name HAVING SUM(sal)>50000;`

NAME	SUM(SAL)
Aarav	125000.75
Aditi	60000.5
Chetan	80000
Divya	220000.75
Gaurav	103000.75

Grouping

- `SELECT NAME, SUM(sal) FROM Emp2 GROUP BY name HAVING SUM(sal)>50000 ORDER BY sum(sal);`

NAME	SUM(SAL)
Aditi	60000.5
Chetan	80000
Gaurav	103000.75
Aarav	125000.75
Divya	220000.75

Grouping

- `SELECT NAME, SUM(sal) FROM Emp2 GROUP BY name HAVING SUM(sal)>50000 ORDER BY sum(sal) DESC;`

NAME	SUM(SAL)
Divya	220000.75
Aarav	125000.75
Gaurav	103000.75
Chetan	80000
Aditi	60000.5

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, **aggregation**, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Aggregation

- An aggregate function is a function that **performs a calculation on a set of values, and returns a single value.**
- Aggregate functions are often used with the **GROUP BY** clause of the SELECT statement.
- The most commonly used SQL aggregate functions are:
 - **MIN()** - returns the smallest value within the selected column
 - **MAX()** - returns the largest value within the selected column
 - **COUNT()** - returns the number of rows in a set
 - **SUM()** - returns the total sum of a numerical column
 - **AVG()** - returns the average value of a numerical column

Aggregation

- Select * from Student2;

ROLL_NUMBER	S_NAME	MARKS	SUBJECT
1	ABHI	70	Maths
2	RAVI	80	English
3	ARJUN	90	Maths
4	SAM	100	English
5	MOHAN	50	Maths
6	ROHAN	10	English
7	ROCKY	20	Science
8	AYUSH	40	Science
9	NEHA	30	Maths
10	KRITI	60	Social

Aggregation: Count()

- select **Count(Subject)** as SubjectCount from Student2;

SUBJECTCOUNT
10

- select **Count(Distinct Subject)** as SubjectCount from Student2;

SUBJECTCOUNT
4

- select **Count(Subject)** as SubjectCount from Student2 where **Subject='Social'**;

SUBJECTCOUNT
1

Aggregation: Sum()

- Select **Sum(marks)** as TotalMarks from Student2;

TOTALMARKS
550

- Select **Subject,Sum(marks)** as TotalMarks from Student2 **Group By (Subject);**

SUBJECT	TOTALMARKS
Maths	240
English	190
Science	60
Social	60

Aggregation: MAX()

- Select **max(marks)** as MaximumMarks from Student2;

MAXIMUMMARKS
100

- Select **Subject,max(marks)** as MaximumMarks from Student2 **group by (Subject);**

SUBJECT	MAXIMUMMARKS
Maths	90
English	100
Science	40
Social	60

Aggregation: Min()

- Select **min(marks)** as MinimumMarks from Student2;

MINUMUMMARKS
10

- Select **Subject,min(marks)** as MinimumMarks from Student2 **group by (Subject);**

SUBJECT	MINUMUMMARKS
Maths	30
English	10
Science	20
Social	60

Aggregation: AVG()

- Select **Avg(marks)** as AverageMarks from Student2;

AVERAGEMARKS

55

- Select **Subject,Avg(marks)** as AverageMarks from Student2 **group by (Subject);**

SUBJECT	AVERAGEMARKS
Maths	60
English	63.333333333333336
Science	30
Social	60

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, **ordering**, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

Ordering

- The ORDER BY clause in SQL is used to sort query results based on one or more columns in either ascending (ASC) or descending (DESC) order.
- **By default, it sorts in ascending order** (lowest to highest).
- To sort in **descending order**, use the **DESC** keyword.
- **Syntax:** SELECT * FROM table_name ORDER BY column_name ASC | DESC;

Ordering

- Select * from Student2;

ROLL_NUMBER	S_NAME	MARKS	SUBJECT
1	ABHI	70	Maths
2	RAVI	80	English
3	ARJUN	90	Maths
4	SAM	100	English
5	MOHAN	50	Maths
6	ROHAN	10	English
7	ROCKY	20	Science
8	AYUSH	40	Science
9	NEHA	30	Maths
10	KRITI	60	Social

Ordering

- ❑ `select * from Student2 order by S_Name;`
- ❑ `select * from Student2 order by S_Name DESC;`
- ❑ `select * from Student2 order by Marks,
Roll_Number DESC;`
- ❑ `select * from Student2 order by S_Name DESC,
Roll_Number;`

UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, **implementation of different types of joins**, view(updatable and non-updatable), relational set operations.

Implementation of different types of joins

- Join is an operation in DBMS that combines the rows of two or more tables based on related columns between them.
- The main purpose of join is to retrieve the data from multiple tables in other words Join is used to perform multi-table queries.

Implementation of different types of joins

□ **Types of Joins:**

□ **Inner Joins:** These joins are the one that has the tuples that satisfy some conditions and rest are discarded.

- Equi join (INNER join), Natural join

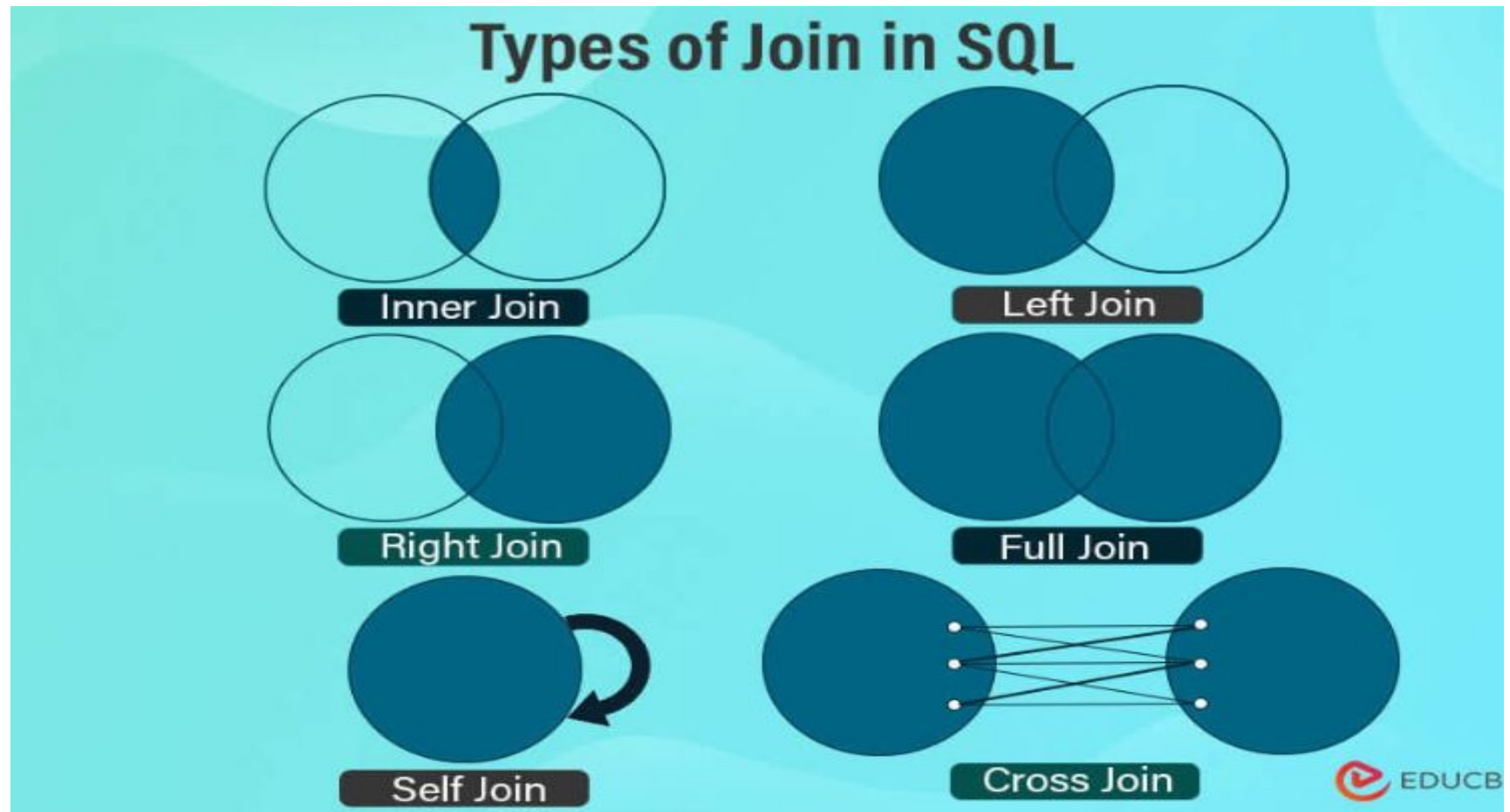
□ **Outer Joins:** These have all the tuples from either or both the relations.

- Left outer join, Right outer join, Full outer join

□ **Cross Joins**

□ **Self Join**

Implementation of different types of joins



Joins:

□ **Join** = **Cross Product** + **Some Condition**

Emp

Dept

E_N O	E_Name	Addres s
1	Ram	Delhi
2	Varun	Hyd
3	Ravi	Hyd
4	Amrut	Pune

DeptNo	Locatio n	E_No
D1	Delhi	1
D2	Pune	2
D3	Patna	4

□ find the names of the employees who is working in a department?

Joins:

□ **Join** = **Cross Product** = $m * n$

m = Number of Rows in Table1 = 4

n = Number of Rows in Table2 = 3

$$4 * 3 = 12$$

E_No	E_Name	Address
1	Ram	Delhi
2	Varun	Hyd
3	Ravi	Hyd
4	Amrut	Pune

DeptNo	Location	E_No
D1	Delhi	1
D2	Pune	2
D3	Patna	4

Joins:

□ **Join = Cross Product**

E_N O	E_Name	Addres s
1	Ram	Delhi
2	Varun	Hyd
3	Ravi	Hyd
4	Amrut	Pune
DeptNo	Locatio n	E_No
D1	Delhi	1
D2	Pune	2
D3	Patna	4

E_NO	E_Nam e	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

□ **Join** = **Cross Product** +

Condition

□ find the names of the employees
who is working in a department?

E_NO	E_Name	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

□ **Join** = **Cross Product** +

Condition

□ find the names of the employees
who is working in a department?

□ **Select E_Name from**

Emp,Dept where

Emp.E_No=Dept.E_No;

E_NO	E_Nam e	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

□ **Join** = **Cross Product** +

Condition

□ find the names of the employees
who is working in a department?

□ **Select E_Name from**

Emp,Dept where

Emp.E_No=Dept.E_No;

E_NO	E_Nam e	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

□ **Join** = **Cross Product** +

Condition

□ find the names of the employees
who is working in a department
And address same as location?

E_NO	E_Name	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

□ **Join** = **Cross Product** +

Condition

□ find the names of the employees
who is working in a department
And address same as location?

□ **Select E_Name from**
Emp,Dept where
Emp.E_No=Dept.E_No
And Emp.Address=
Dept.Location;

E_NO	E_Name	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

□ **Join** = **Cross Product** +

Condition

□ find the names of the employees
who is working in a department
And address same as location?

□ **Select E_Name from**
Emp,Dept where
Emp.E_No=Dept.E_No
And Emp.Address=
Dept.Location;

E_NO	E_Name	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

□ **Join** = **Cross Product** +

Condition

□ find the names of the employees
who is working in a department?

□ **Select E_Name from**

Emp,Dept where

Emp.E_No=Dept.E_No

And Emp.Address=

Dept.Location;

E_NO	E_Nam e	Address	Dept No	Location	E_No
1	Ram	Delhi	D1	Delhi	1
1	Ram	Delhi	D2	Pune	2
1	Ram	Delhi	D3	Patna	4
2	Varun	Hyd	D1	Delhi	1
2	Varun	Hyd	D2	Pune	2
2	Varun	Hyd	D3	Patna	4
3	Ravi	Hyd	D1	Delhi	1
3	Ravi	Hyd	D2	Pune	2
3	Ravi	Hyd	D3	Patna	4
4	Amrut	Pune	D1	Delhi	1
4	Amrut	Pune	D2	Pune	2
4	Amrut	Pune	D3	Patna	4

Joins:

- Find the student id who enrolled for two courses?
- **Study:**

S_ID	C_ID	Since
S1	C1	2023
S2	C2	2023
S1	C2	2024

Joins:

- Find the student id who enrolled for two courses?

S_ID	C_ID	Since
S1	C1	2023
S2	C2	2023
S1	C2	2024

S_ID	C_ID	Since
S1	C1	2023
S2	C2	2023
S1	C2	2024

S_ID	C_ID	Since	S_ID	C_ID	Since
S1	C1	2023	S1	C1	2023
S1	C1	2023	S2	C2	2023
S1	C1	2023	S1	C2	2024
S2	C2	2023	S1	C1	2023
S2	C2	2023	S2	C2	2023
S2	C2	2023	S1	C2	2024
S1	C2	2024	S1	C1	2023
S1	C2	2024	S2	C2	2023
S1	C2	2024	S1	C2	2024

Joins:

□ Find the student id who enrolled for two courses?

□ Select T1.S_ID from
Study as T1, Study as T2
Where **T1.S_ID = T2.S_ID**
And
T1.C_ID <> T2.C_ID;

S_ID	C_ID	Since	S_ID	C_ID	Since
S1	C1	2023	S1	C1	2023
S1	C1	2023	S2	C2	2023
S1	C1	2023	S1	C2	2024
S2	C2	2023	S1	C1	2023
S2	C2	2023	S2	C2	2023
S2	C2	2023	S1	C2	2024
S1	C2	2024	S1	C1	2023
S1	C2	2024	S2	C2	2023
S1	C2	2024	S1	C2	2024

Joins:

□ Find the student id who enrolled for two courses?

□ Select T1.S_ID from
Study as T1, Study as T2
Where **T1.S_ID = T2.S_ID**
And
T1.C_ID <> T2.C_ID;

S_ID	C_ID	Since	S_ID	C_ID	Since
S1	C1	2023	S1	C1	2023
S1	C1	2023	S2	C2	2023
S1	C1	2023	S1	C2	2024
S2	C2	2023	S1	C1	2023
S2	C2	2023	S2	C2	2023
S2	C2	2023	S1	C2	2024
S1	C2	2024	S1	C1	2023
S1	C2	2024	S2	C2	2023
S1	C2	2024	S1	C2	2024

Implementation of different types of joins

- ❑ Create Table EmpJ(Emp_Id int, Emp_Name varchar(30), Emp_Phone Int);
- ❑ INSERT INTO EmpJ (EMP_Id, Emp_name, Emp_Phone) VALUES
(1, 'Aarav', 1111),
(2, 'Bhanu', 2222),
(3, 'Charan', 3333),
(4, 'Dhanush', 4444),
(5, 'Geetha', 5555);

Implementation of different types of joins

- Create Table SalJ(Emp_Id int, T_ID Int, Amount Int);
- INSERT INTO SalJ(EMP_Id, T_id, Amount) VALUES
(1, 101, 2000),
(2, 202, 2500),
(3, 303, 3000),
(4, 404, 4500),
(6, 606, 6500);

Implementation of different types of joins

EmpJ

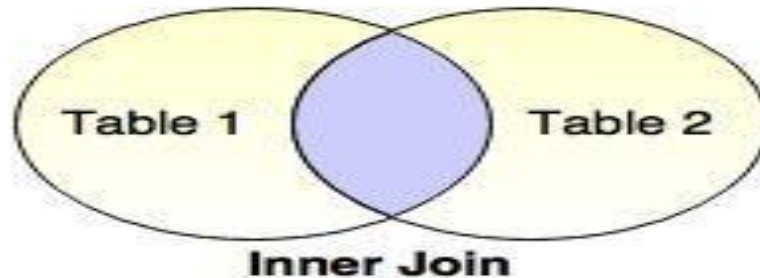
EMP_ID	EMP_NAME	EMP_PHONE
1	Aarav	1111
2	Bhanu	2222
3	Charan	3333
4	Dhanush	4444
5	Geetha	5555

SalJ

EMP_ID	T_ID	AMOUNT
1	101	2000
2	202	2500
3	303	3000
4	404	4500
6	606	6500

Implementation of different types of joins- **Inner Join**

- If there is a **match between the columns** in both tables, SQL INNER JOIN statement **selects all possible rows from both tables**.
- If there is **no match between columns** in both columns, the **data should not be displayed**.



Implementation of different types of joins- **Inner Join**

```
□ SELECT column_name(s)  
  FROM table1 INNER JOIN table2  
  ON table1.column_name = table2.column_name;
```

Implementation of different types of joins- Inner Join

```
□ select E.Emp_Id, E.Emp_Name, S.Amount  
from EmpJ E Inner Join SalJ S  
On E.Emp_Id = S.Emp_Id;
```

EMP_ID	EMP_NAME	AMOUNT
1	Aarav	2000
2	Bhanu	2500
3	Charan	3000
4	Dhanush	4500

Implementation of different types of joins- **Natural Join**

- Natural Join is a **type of Inner join** which is based on **column having same name and same datatype** present in both the tables to be joined.
- The syntax for Natural Join is:
 - `SELECT * FROM tablename1 NATURAL JOIN table name2;`

Implementation of different types of joins- **Natural Join**

Here is the **class** table,

ID	NAME
1	abhi
2	adam
3	alex
4	anu

and the **class_info** table,

ID	Address
1	DELHI
2	MUMBAI
3	CHENNAI

Implementation of different types of joins- **Natural Join**

- Natural join query will be,

- `SELECT * from class NATURAL JOIN class_info;`

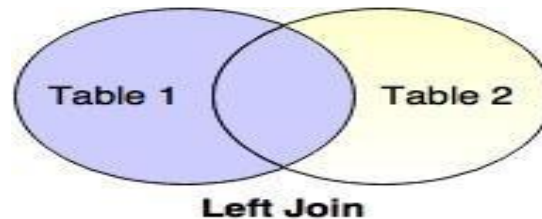
The resultset table will look like,

ID	NAME	Address
1	abhi	DELHI
2	adam	MUMBAI
3	alex	CHENNAI

- In the above example, both the tables being joined have ID column(same name and same datatype)

Implementation of different types of joins- **Left Join**

- A SQL LEFT JOIN, also known as a LEFT OUTER JOIN, is a type of SQL JOIN operation that combines rows from two tables based on a related column between them.
- Key characteristics of a LEFT JOIN:
 - Returns all rows from the left table.
 - Returns matching rows from the right table.
 - Includes NULL for non-matching rows in the right table.



Implementation of different types of joins- **Left Join**

- Syntax:
- `SELECT column_name`
`FROM table1`
`LEFT JOIN table2`
`ON table1.column_name = table2.column_name;`

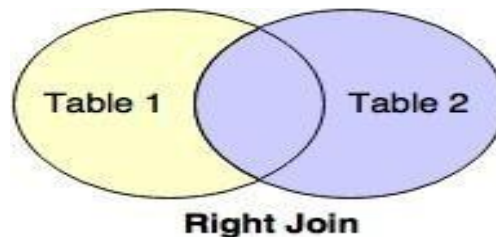
Implementation of different types of joins- **Left Join**

- `select E.Emp_Id, E.Emp_Name, S.Amount`
`from EmpJ E Left Join SalJ S`
`On E.Emp_Id = S.Emp_Id;`

EMP_ID	EMP_NAME	AMOUNT
1	Aarav	2000
2	Bhanu	2500
3	Charan	3000
4	Dhanush	4500
5	Geetha	(null)

Implementation of different types of joins- **Right Join**

- A SQL RIGHT JOIN, also known as a RIGHT OUTER JOIN, is a type of SQL JOIN operation that combines rows from two or more tables based on a related column.
- Key characteristics of a RIGHT JOIN:
 - Returns all rows from the Right table.
 - Returns matching rows from the Left table.
 - Includes NULL for non-matching rows in the Left table.



Implementation of different types of joins- **Right Join**

```
□ select E.Emp_Id, E.Emp_Name, S.Amount  
from EmpJ E Right Join SalJ S  
On E.Emp_Id = S.Emp_Id;
```

EMP_ID	EMP_NAME	AMOUNT
1	Aarav	2000
2	Bhanu	2500
3	Charan	3000
4	Dhanush	4500
(null)	(null)	6500

Implementation of different types of joins- **Full Join**

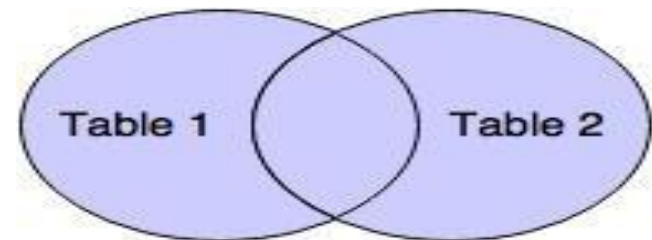
- The FULL JOIN or FULL OUTER JOIN statement returns all rows from the left table (table1) and the right table (table2).

□ SELECT column_name

FROM table1

FULL JOIN table2

ON table1.column_name = table2.column_name;



Full Outer Join

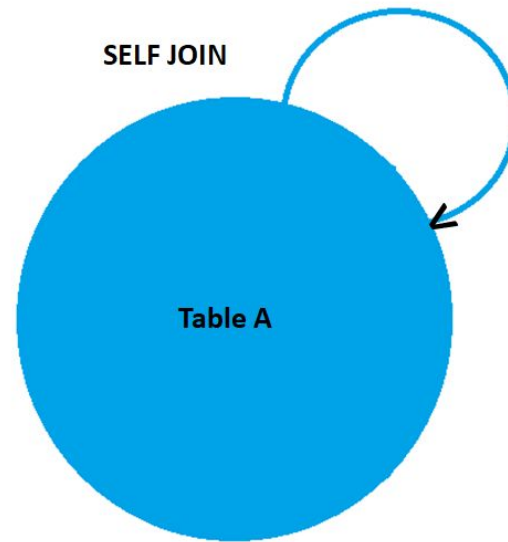
Implementation of different types of joins- Full Join

```
□ select E.Emp_Id, E.Emp_Name, S.Amount  
from EmpJ E Full Join SalJ S  
On E.Emp_Id = S.Emp_Id;
```

EMP_ID	EMP_NAME	AMOUNT
1	Aarav	2000
2	Bhanu	2500
3	Charan	3000
4	Dhanush	4500
(null)	(null)	6500
5	Geetha	(null)

Implementation of different types of joins: **Self Join**

- A self-join in SQL is a join where **a table is joined with itself.**



Implementation of different types of joins: **Self Join**

- ❑ `CREATE TABLE GFGEmployees1 (
employee_id INT PRIMARY KEY,
employee_name VARCHAR(50),
manager_id INT);`
- ❑ `INSERT INTO GFGEmployees1 (employee_id,
employee_name, manager_id) VALUES
(1, 'Zaid', 3),
(2, 'Rahul', 3),
(3, 'Raman', 4),
(4, 'Kamran', NULL),
(5, 'Farhan', 4);`

Implementation of different types of joins: Self Join

- Select * from GFGEmployees1;

EMPLOYEE_ID	EMPLOYEE_NAME	MANAGER_ID
1	Zaid	3
2	Rahul	3
3	Raman	4
4	Kamran	(null)
5	Farhan	4

Implementation of different types of joins: Self Join

- Find the employees names and their corresponding managers names?
- ```
SELECT E.employee_name AS Employee,
M.employee_name AS Manager
FROM GFGemployees1 E
JOIN GFGemployees1 M ON
E.manager_id = M.employee_id;
```

| EMPLOYEE | MANAGER |
|----------|---------|
| Zaid     | Raman   |
| Rahul    | Raman   |
| Raman    | Kamran  |
| Farhan   | Kamran  |

# UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, **view(updatable and non-updatable)**, relational set operations.

# View(updatable and non-updatable)

---

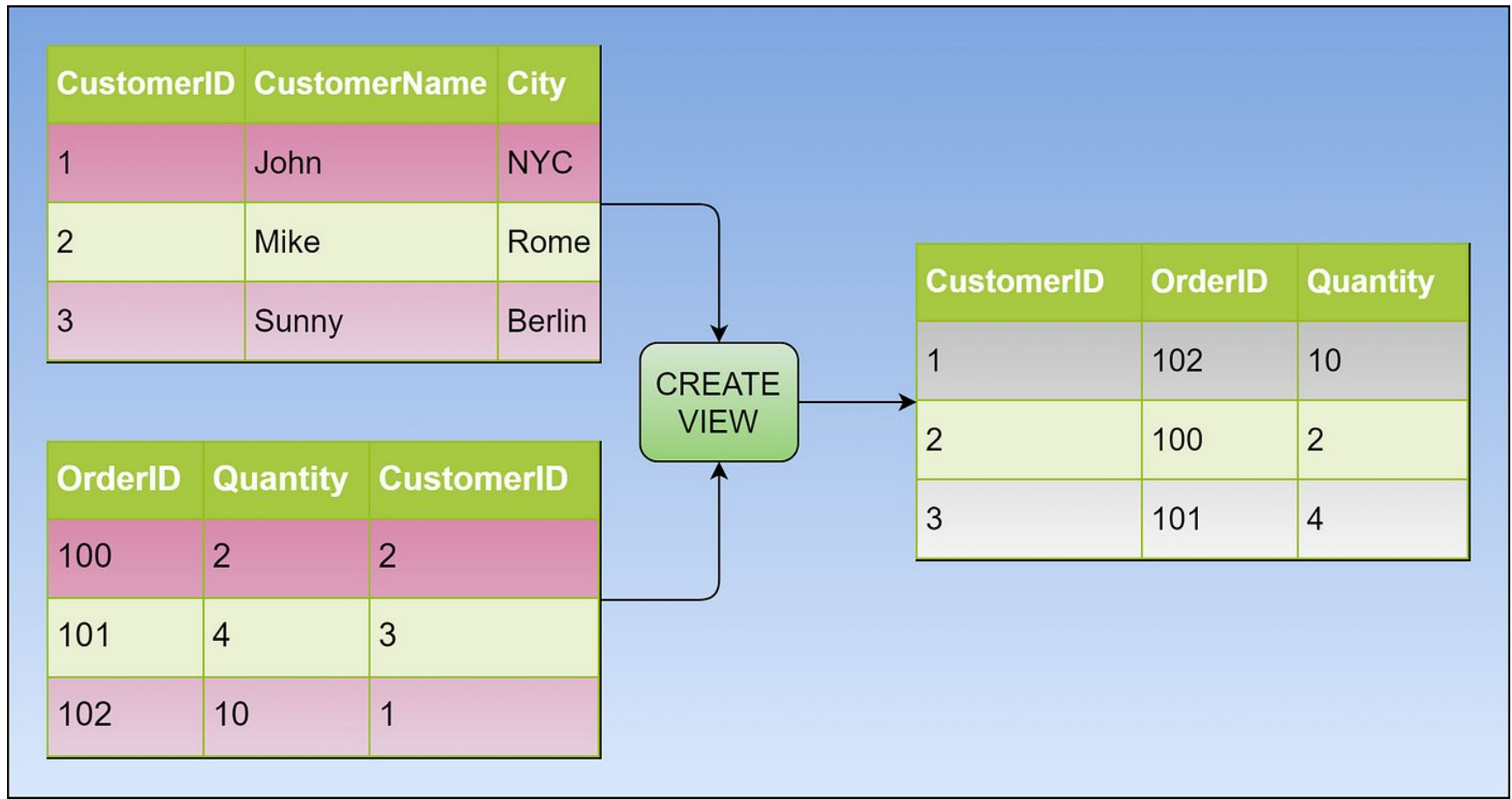
- Introduction
- Examples
- Advantages

# View(updatable and non-updatable)

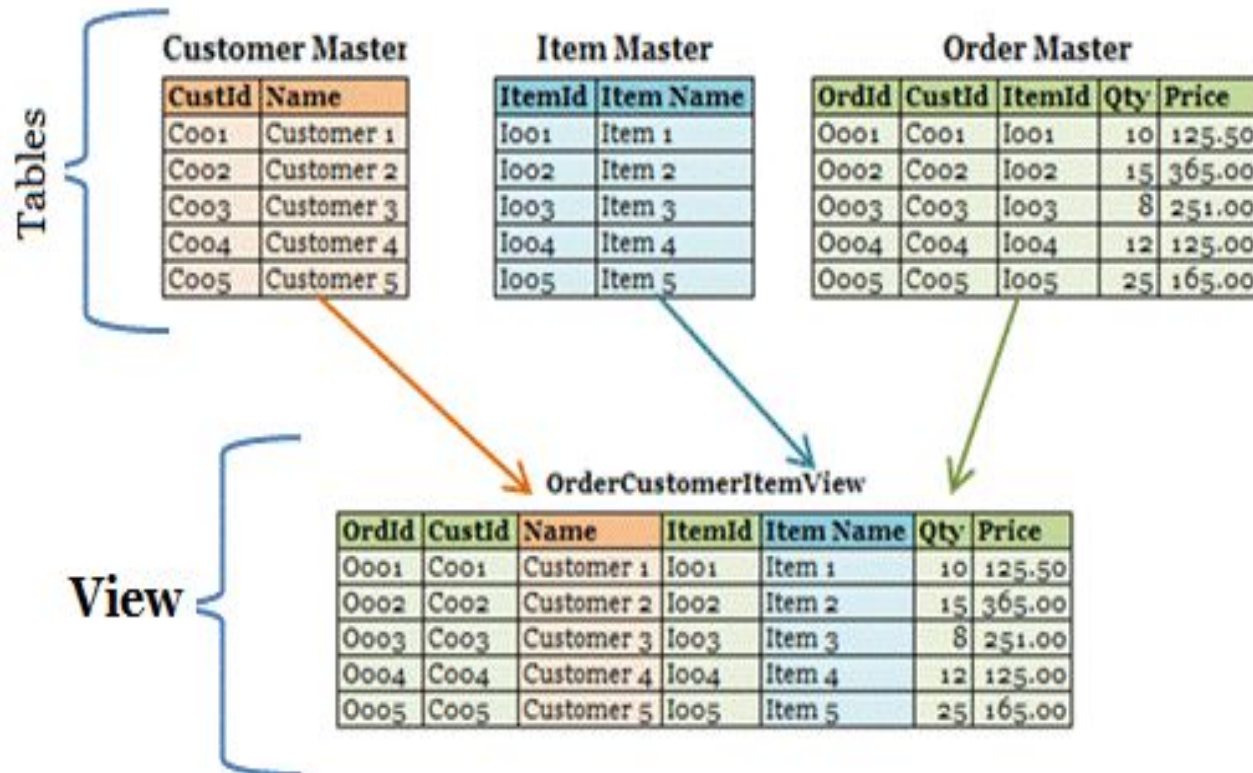
---

- Introduction
- Examples
- Advantages

# View(updatable and non-updatable)



# View(updatable and non-updatable)



# View(updatable and non-updatable)

## Employee

| EmployeeID | Ename | DeptID | Salary |
|------------|-------|--------|--------|
| 1001       | John  | 2      | 4000   |
| 1002       | Anna  | 1      | 3500   |
| 1003       | James | 1      | 2500   |
| 1004       | David | 2      | 5000   |
| 1005       | Mark  | 2      | 3000   |
| 1006       | Steve | 3      | 4500   |
| 1007       | Alice | 3      | 3500   |

```
CREATE VIEW emp_view AS
SELECT EmployeeID, Ename
FROM Employee
WHERE DeptID=2;
```

Creating View  
by filtering  
records using  
WHERE clause

## emp\_view

| EmployeeID | Ename |
|------------|-------|
| 1001       | John  |
| 1004       | David |
| 1005       | Mark  |



VISHNU  
UNIVERSAL LEARNING

# VISHNU INSTITUTE OF TECHNOLOGY

( Approved by AICTE, New Delhi | Affiliated to JNTUK )

Vishnupur, Bhimavaram, West Godavari Dist. - 534 202 Ph : 08816 251333, 250819 Fax : 08816 250344

e-mail : info@vishnuit.in website : www.vishnuit.in

Welcome to Freshers

» ADMIN » EXAMINATIONS » LIBRARY  
» ACADEMICS » CORRESPONDENCE » GENERAL

## ACADEMICS

ABSENTIES REPORT

ACADAMIC REGISTER

ATTENDANCE REPORTS »

CURRENT TIME TABLE

FACULTY »

FACULTY ADJUSTMENTS »

PROJECTS »

RESOURCES

STUDENT »

STUDENT PROFILE

TEACHING PLAN

TEACHING PLAN VERIFY

TEACHING SCHEDULE

TIME TABLE »

UPLOAD RESOURCE

Welcome MR.K.SUDHAKAR

[Change Password](#)

[Log Out](#)

## CLASS ATTENDANCE

Date : 09/09/2025

Course: B.Tech

Semester: 2/4 Semester-I

Branch: Information Technology

Section4

Show

☒ Check Absentees ☐ Check Presentees

| SL.NO | STUDENT NAME             | ROLL.NO    | PERIOD1 DBMS             |
|-------|--------------------------|------------|--------------------------|
|       |                          |            | <input type="checkbox"/> |
| 1     | PULI HADHASSA            | 24PA1A12J3 | <input type="checkbox"/> |
| 2     | PULLE RASAJNA            | 24PA1A12J4 | <input type="checkbox"/> |
| 3     | PURELLA BALA KRISHNA     | 24PA1A12J5 | <input type="checkbox"/> |
| 4     | PUTTA LAHARI             | 24PA1A12J6 | <input type="checkbox"/> |
| 5     | RAMAVATH THIRUMALA NAIK  | 24PA1A12J7 | <input type="checkbox"/> |
| 6     | RANDHI SASI MITHUL       | 24PA1A12J8 | <input type="checkbox"/> |
| 7     | RATHOD SWAPNA            | 24PA1A12J9 | <input type="checkbox"/> |
| 8     | RAVIPATI REKHA ANNAPURNA | 24PA1A12K0 | <input type="checkbox"/> |
| 9     | RAVIPUDI PAVAN KUMAR     | 24PA1A12K1 | <input type="checkbox"/> |
| 10    | RAVURI TEJASWINI         | 24PA1A12K2 | <input type="checkbox"/> |
| 11    | REDDI PAVAN              | 24PA1A12K3 | <input type="checkbox"/> |
| 12    | REDDI PURUSHOTTAM NAIDU  | 24PA1A12K4 | <input type="checkbox"/> |

|    |                                          |            |                          |
|----|------------------------------------------|------------|--------------------------|
| 47 | VASABATTULA V V N S J S S SHANKAR        | 24PA1A12N9 | <input type="checkbox"/> |
| 48 | VASKURI SAI RAMA SRI                     | 24PA1A12O0 | <input type="checkbox"/> |
| 49 | VEDULLAPALLI SAI VARSHITH                | 24PA1A12O1 | <input type="checkbox"/> |
| 50 | VEERAMALLU NIKHITHA                      | 24PA1A12O2 | <input type="checkbox"/> |
| 51 | VEERAVALLI PUJITHA                       | 24PA1A12O3 | <input type="checkbox"/> |
| 52 | VEGESNA YASWANTH KUMAR VARMA             | 24PA1A12O4 | <input type="checkbox"/> |
| 53 | VELAVALAPALLI NAGA SIVA ANJANA SOWMYA    | 24PA1A12O5 | <input type="checkbox"/> |
| 54 | VEMANAMANDHA CHARAN SAI KOWSHIK VARMA    | 24PA1A12O6 | <input type="checkbox"/> |
| 55 | VETUKURI HEMA                            | 24PA1A12O7 | <input type="checkbox"/> |
| 56 | VILLA ANIL NANDHA KARTHIKEYA             | 24PA1A12O8 | <input type="checkbox"/> |
| 57 | VISWANATHARAO NANDA BABU                 | 24PA1A12O9 | <input type="checkbox"/> |
| 58 | VOBILINENI ROHITH SAI DURGA              | 24PA1A12P0 | <input type="checkbox"/> |
| 59 | YALAMANCHI ABHILASH                      | 24PA1A12P1 | <input type="checkbox"/> |
| 60 | YAMBULURI MANOHAR                        | 24PA1A12P2 | <input type="checkbox"/> |
| 61 | YARAKALA SURYA LAKSHMI SAI BHAVITHA      | 24PA1A12P3 | <input type="checkbox"/> |
| 62 | YARRAMSETTI S S S P VAMSI                | 24PA1A12P4 | <input type="checkbox"/> |
| 63 | YELLAPRAGADA SAI VASUDEVA PAVAN YASWANTH | 24PA1A12P5 | <input type="checkbox"/> |
| 64 | PALLI DEEPIKA                            | 25PA5A1225 | <input type="checkbox"/> |
| 65 | PAMARTHI ROHITH VENKATA SAI              | 25PA5A1226 | <input type="checkbox"/> |
| 66 | PILLI SRAVANI                            | 25PA5A1227 | <input type="checkbox"/> |
| 67 | SANABOYINA GANESH                        | 25PA5A1228 | <input type="checkbox"/> |
| 68 | SHAIK MUFEED                             | 25PA5A1229 | <input type="checkbox"/> |
| 69 | TANUKU JESHWANTH SATHYA GANESH           | 25PA5A1230 | <input type="checkbox"/> |
| 70 | THALLURI SATHYA SAI                      | 25PA5A1231 | <input type="checkbox"/> |
| 71 | ULLAMPARTHI SUBASH BABU                  | 25PA5A1232 | <input type="checkbox"/> |
| 72 | VADDI HARSHITHA                          | 25PA5A1233 | <input type="checkbox"/> |

| TOPICS COVERED                                                            |           |                      |                      |
|---------------------------------------------------------------------------|-----------|----------------------|----------------------|
| SUBJECT                                                                   | PERIOD.NO | UNIT                 | TOPICS COVERED       |
| DBMS                                                                      | 1         | <input type="text"/> | <input type="text"/> |
| <input type="button" value="Save"/> <input type="button" value="Cancel"/> |           |                      |                      |

# View(updatable and non-updatable)

---

- Introduction
- Examples

# View(updatable and non-updatable)

---

- Introduction
- Examples

# View(updatable and non-updatable)

- A view in SQL is a **saved SQL query that acts as a virtual table**.
- It can **fetch data from one or more tables** and present it in a customized format.
- Unlike regular tables, **views do not store data themselves**.
- Instead, they **dynamically generate** data by executing the SQL query defined in the view **each time it is accessed**.

# View(updatable and non-updatable)

## □ Non-Updatable Views (Read-Only Views):

- A non-updatable view, also known as a **read-only** view, only allows data retrieval (**SELECT** operations).
- **INSERT, UPDATE, and DELETE** operations are not permitted directly on the view.

## □ Updatable Views:

- An updatable view **allows INSERT, UPDATE, and DELETE** operations to be performed directly on the view.
- These **changes** are then propagated to the underlying base **table(s)**.

# View(updatable and non-updatable)

- **Uses and Benefits of SQL Views:**
- **Restricting data access** - Views provide an additional level of table security by restricting access to a predetermined set of rows and columns of a table.
- **Simplify commands for the user** - Views allow the user to select information from multiple tables without requiring the users to actually know how to perform a join.
- **Multiple view facility** - Different views can be created on the same table for different users.

# View(updatable and non-updatable)

---

- Introduction
- Examples

# View(**updatable** and non-updatable)

```
❑ CREATE TABLE CUSTOMERS(
 ID INT NOT NULL,
 NAME VARCHAR (20) NOT NULL,
 AGE INT NOT NULL,
 ADDRESS CHAR (25) ,
 SALARY DECIMAL (18, 2),
 PRIMARY KEY (ID)
);
```

# View(**updatable** and non-updatable)

## ❑ INSERT INTO CUSTOMERS VALUES

```
(1, 'Ramesh', 32, 'Ahmedabad', 2000.00),
(2, 'Khilan', 25, 'Delhi', 1500.00),
(3, 'Kaushik', 23, 'Kota', 2000.00),
(4, 'Chaitali', 25, 'Mumbai', 6500.00),
(5, 'Hardik', 27, 'Bhopal', 8500.00),
(6, 'Komal', 22, 'Hyderabad', 4500.00),
(7, 'Muffy', 24, 'Indore', 10000.00);
```

# View(**updatable** and non-updatable)

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 32  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 3  | Kaushik  | 23  | Kota      | 2000   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |
| 6  | Komal    | 22  | Hyderabad | 4500   |
| 7  | Muffy    | 24  | Indore    | 10000  |

# View(**updatable** and non-updatable)

- **create view** CustView as select Name, Age, Salary from Customers;
- **\*/create view** CustView as select \* from Customers; **\*/**

| NAME     | AGE | SALARY |
|----------|-----|--------|
| Ramesh   | 32  | 2000   |
| Khilan   | 25  | 1500   |
| Kaushik  | 23  | 2000   |
| Chaitali | 25  | 6500   |
| Hardik   | 27  | 8500   |
| Komal    | 22  | 4500   |
| Muffy    | 24  | 10000  |

# View(**updatable** and non-updatable)

- Select name from CustView;

| NAME     |
|----------|
| Ramesh   |
| Khilan   |
| Kaushik  |
| Chaitali |
| Hardik   |
| Komal    |
| Muffy    |

# View(**updatable** and non-updatable)

- **create** view Custview1 as select \* from Customers where age>24;
- Select \* from Custview1;

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 32  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |

# View(**updatable** and non-updatable)

- **UPDATE** Custview1 SET Age=40 where Name='Ramesh';
- **select \* from CustView1;**

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |

# View(**updatable** and non-updatable)

□ `select * from Customers;`

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 3  | Kaushik  | 23  | Kota      | 2000   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |
| 6  | Komal    | 22  | Hyderabad | 4500   |
| 7  | Muffy    | 24  | Indore    | 10000  |

# View(**updatable** and non-updatable)

- **insert** into Custview1 values (8,'Satish',29,'Delhi',8000);
- **select \*** from Custview1;

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |
| 8  | Satish   | 29  | Delhi     | 8000   |

# View(**updatable** and non-updatable)

□ `select * from Customers;`

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 3  | Kaushik  | 23  | Kota      | 2000   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |
| 6  | Komal    | 22  | Hyderabad | 4500   |
| 7  | Muffy    | 24  | Indore    | 10000  |
| 8  | Satish   | 29  | Delhi     | 8000   |

# View(**updatable** and non-updatable)

- ❑ **delete** from Custview1 where name='Satish';
- ❑ Select \* from Custview1;

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |

# View(**updatable** and non-updatable)

□ `select * from Customers;`

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 3  | Kaushik  | 23  | Kota      | 2000   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |
| 6  | Komal    | 22  | Hyderabad | 4500   |
| 7  | Muffy    | 24  | Indore    | 10000  |

# View(**updatable** and non-updatable)

- **delete** from Customers where name='Muffy';

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 3  | Kaushik  | 23  | Kota      | 2000   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |
| 6  | Komal    | 22  | Hyderabad | 4500   |

# View(**updatable** and non-updatable)

- `select * from Custview1;`

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 5  | Hardik   | 27  | Bhopal    | 8500   |

# View(**updatable** and non-updatable)

- **delete** from Customers where name='Hardik';
- **select \*** from Customers;

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 3  | Kaushik  | 23  | Kota      | 2000   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |
| 6  | Komal    | 22  | Hyderabad | 4500   |

# View(**updatable** and non-updatable)

□ `select * from Custview1;`

| ID | NAME     | AGE | ADDRESS   | SALARY |
|----|----------|-----|-----------|--------|
| 1  | Ramesh   | 40  | Ahmedabad | 2000   |
| 2  | Khilan   | 25  | Delhi     | 1500   |
| 4  | Chaitali | 25  | Mumbai    | 6500   |

# View(**updatable** and non-updatable)

- ❑ CREATE TABLE CUSTOMERSROLE(  
ID INT NOT NULL,  
Department VARCHAR(20) NOT NULL,  
Jobrole VARCHAR(25));

# View(**updatable** and non-updatable)

## ❑ INSERT INTO CUSTOMERSROLE VALUES

```
(1, 'Accounts', 'Manager'),
(2, 'Sales', 'Accountant'),
(3, 'Marketing','Head'),
(4, 'Accounts', 'Employee'),
(5, 'Sales', 'Manager'),
(6, 'Marketing','Advertising'),
(7, 'Sales', 'Asst.MAnager');
```

# View(**updatable** and non-updatable)

- `select * from CustomersRole;`

| ID | DEPARTMENT | JOBROLE      |
|----|------------|--------------|
| 1  | Accounts   | Manager      |
| 2  | Sales      | Accountant   |
| 3  | Marketing  | Head         |
| 4  | Accounts   | Employee     |
| 5  | Sales      | Manager      |
| 6  | Marketing  | Advertising  |
| 7  | Sales      | Asst.MAnager |

# View(**updatable** and non-updatable)

- **Views from Multiple Tables:**

- create view CustDetails AS

Select C.Id, C.Name, C.Salary, CR.Department,  
CR.Jobrole

from Customers C, CustomersRole CR

Where C.ID=CR.ID;

# View(**updatable** and non-updatable)

- Select \* from CustDetails;

| ID | NAME     | SALARY | DEPARTMENT | JOBROLE     |
|----|----------|--------|------------|-------------|
| 1  | Ramesh   | 2000   | Accounts   | Manager     |
| 2  | Khilan   | 1500   | Sales      | Accountant  |
| 3  | Kaushik  | 2000   | Marketing  | Head        |
| 4  | Chaitali | 6500   | Accounts   | Employee    |
| 6  | Komal    | 4500   | Marketing  | Advertising |

# View(updatable and non-updatable)

- There are **two primary methods** for creating a non-updatable view in SQL:
  - **Revoking modification permissions** from users.
  - Defining the **view with complex logic**.
- A non-updatable view is also known as a **read-only view**.
- **Method 1:** Revoking modification permissions from users.
  - **Grant** Select on Customers to Users;

# View(updatable and non-updatable)

- ❑ **Method 2:** Defining the view with complex logic/ Use non-updatable query constructs
- ❑ When creating a view, include functions or clauses that inherently make the view non-updatable.
- ❑ The database will then prevent INSERT, UPDATE, and DELETE operations on it because it cannot determine which row in the underlying table(s) to modify.

# View(updatable and non-updatable)

- Select \* from Student2;

| ROLL_NUMBER | S_NAME | MARKS | SUBJECT |
|-------------|--------|-------|---------|
| 1           | ABHI   | 70    | Maths   |
| 2           | RAVI   | 80    | English |
| 3           | ARJUN  | 90    | Maths   |
| 4           | SAM    | 100   | English |
| 5           | MOHAN  | 50    | Maths   |
| 6           | ROHAN  | 10    | English |
| 7           | ROCKY  | 20    | Science |
| 8           | AYUSH  | 40    | Science |
| 9           | NEHA   | 30    | Maths   |
| 10          | KRITI  | 60    | Social  |

# View(updatable and non-updatable)

- Create View NonUpdateView As  
select Sum(Marks) as TotalMarks, Subject from Student2 Group  
By (Subject);
- Select \* from NonUpdateView;

| TOTALMARKS | SUBJECT |
|------------|---------|
| 240        | Maths   |
| 190        | English |
| 60         | Science |
| 60         | Social  |

# View(updatable and **non-updatable**)

- update nonupdateview set totalmarks = 230 where subject='Maths';

```
SQL> update nonupdateview set totalmarks = 230 where subject='Maths'
```

```
ORA-01732: data manipulation operation not legal on this view
```

```
https://docs.oracle.com/error-help/db/ora-01732/
```

```
Error at Line: 4 Column: 0
```

---

# UNIT III:

- SQL: Basic SQL querying (select and project) using where clause, arithmetic & logical operations, SQL functions(Date and Time, Numeric, String conversion).Creating tables with relationship, implementation of key and integrity constraints, nested queries, sub queries, grouping, aggregation, ordering, implementation of different types of joins, view(updatable and non-updatable), relational set operations.

# Relational set operations

- SQL supports few Set operations which can be performed on the table data.
- These are used to get meaningful results from data stored in the table, under different special conditions.
- The different SET operations are:
  - UNION
  - UNION ALL
  - INTERSECT
  - MINUS

# Relational set operations

- create table **Table1**(id int, name varchar(20));
- insert into Table1 (id,name) VALUES  
(1,'Arun'),  
(2,'Bhanu') ;
- create table **Table2**(id int, name varchar(20));
- insert into Table2 (id,name) VALUES  
(2,'Bhanu'),  
(3,'Chandu');

# Relational set operations

- `select * from Table1;`

| ID | NAME  |
|----|-------|
| 1  | Arun  |
| 2  | Bhanu |

- `select * from Table2;`

| ID | NAME   |
|----|--------|
| 2  | Bhanu  |
| 3  | Chandu |

# Relational set operations: Union

- ❑ **UNION** is used to combine the results of two or more SELECT statements.
- ❑ However it will eliminate duplicate rows from its result set.
- ❑ In case of union, number of columns and datatype must be same in both the tables, on which UNION operation is being applied.

# Relational set operations: Union

- Select \* from Table1

Union

Select \* from Table2;

| ID | NAME   |
|----|--------|
| 1  | Arun   |
| 2  | Bhanu  |
| 3  | Chandu |

# Relational set operations: Union ALL

- This operation is similar to Union. But it also shows the duplicate rows.
- `Select * from Table1 Union All Select * from Table2;`

| ID | NAME   |
|----|--------|
| 1  | Arun   |
| 2  | Bhanu  |
| 2  | Bhanu  |
| 3  | Chandu |

# Relational set operations: **Intersect**

- Intersect operation is used to combine two SELECT statements, but it only returns the records which are common from both SELECT statements.
- `Select * from Table1 INTERSECT Select * from Table2;`

| ID | NAME  |
|----|-------|
| 2  | Bhanu |

# Relational set operations: MINUS

- The Minus operation combines results of two SELECT statements and return only those in the final result, which belongs to the first set of the result.
- `Select * from Table1 MINUS Select * from Table2;`

| ID | NAME |
|----|------|
| 1  | Arun |